

Backend(Sunday 08/03/226) :

Pagination.py : to create a reusable pagination class that controls how many results your API returns at once and when a client requests data from my API like a list of trips and instead of returning all records at once this code will splits them into smaller pages

```
# Without pagination - returns EVERYTHING (bad!)
GET /api/trips/ → Returns 10,000 trips (slow, crashes easily)
```

```
# With pagination - returns just one page at a time
GET /api/trips/ → Returns first 10 trips + "next page" link
GET /api/trips/?page=2 → Returns next 10 trips
```

Setting	Value	Purpose
page_size	10	Default: show 10 items per page
page_size_query_param	"page_size"	Allow clients to change page size via ?page_size=25
max_page_size	100	Maximum allowed items per page (security)

This pagination class prevents your API from crashing by limiting results to 10 items per page (customizable up to 100), with automatic "next/previous" links for navigation

Permissions.py: these three permission classes control who can access different parts of your API and it will be :

Permission 1: **IsOwnerOrReadOnly**

```
# User A created Trip #42
```

```
# Anyone can view it
GET /api/trips/42 →  ALLOWED
```

```
# Only User A can edit it
PUT /api/trips/42 (as User A) →  ALLOWED
PUT /api/trips/42 (as User B) →  FORBIDDEN
DELETE /api/trips/42 (as anyone else) →  FORBIDDEN
```

By this anyone can view GET the object and only the owner can modify it
OUT PATCH DELETE

Permission 2: **IsAdminOrReadOnly**

Regular users

GET /api/settings/ →  ALLOWED

POST /api/settings/ →  FORBIDDEN

Admin users

GET /api/settings/ →  ALLOWED

POST /api/settings/ →  ALLOWED

By this anyone can view GET content and only the admin STAFF USERS can
modify it

Permission 3: **IsAuthenticatedOrCreateOnly**

New user registration

POST /api/users/ (no login) →  ALLOWED

Other operations

GET /api/profile/ (no login) →  FORBIDDEN

PUT /api/profile/ (logged in) →  ALLOWED

DELETE /api/posts/1 (logged in) →  ALLOWED

By this anyone can create new accounts POST and only logged-in users can
anything else

Permission	Read (GET)	Create (POST)	Update (PUT)	Delete	Special Rule
IsOwnerOrReadOnly	 Anyone	 Owner	 Owner	 Owner	Owner = object.user
IsAdminOrReadOnly	 Anyone	 Admin only	 Admin only	 Admin only	Admin = is_staff
IsAuthenticatedOrCreateOnly	 Auth only	 Anyone	 Auth only	 Auth only	Only POST create is public

These permissions control API access by allowing anyone to view content but restricting modifications to owners, admins, or authenticated users depending on the need.

Signals.py : this code automatically sends notifications when important events happen in our carpooling app and every signal will :

New Booking → Alert Driver

When a passenger books a trip:

John books a seat on Karim's trip to ALGER

Automatically:

 Karim gets: "John wants to join your trip to ALGER"

New Rating → Alert Rated User

When someone rates a user:

Sarah gives Mike 5 stars

Automatically:

 Mike gets: "Sarah rated you 5/5"

Trip Cancelled → Alert All Passengers

When a driver cancels their trip:

Karim cancels his Algiers→Oran trip (5 passengers booked)

Automatically:

 All 5 passengers get: "Trip Algiers→Oran has been cancelled"

New User → Welcome Message

When someone creates an account:

Ahmed signs up

Automatically:

 Ahmed gets: "Welcome to SHARETRAJET, Ahmed! Complete your profile."

Field	Purpose	Example
-------	---------	---------

recipient	Who gets the notification	The driver, the rated user
sender	Who triggered the action	The passenger, the rater
type	What kind of event	RESERVATION_REQUEST, RATING_RECEIVED
content	The message	"John wants to join your trip"
related_model	What this refers to	"Reservation", "Rating", "Trajet"
related_id	ID of the related object	42

Throttling.py : this code limits how many requests users can make to prevent abuse and ensure fair usage of my API

BurstRateThrottle - For Logged-in Users

Maximum 60 requests per minute and prevents a user from flooding my API with too many requests too quickly

SustainedRateThrottle - Daily Limit

Maximum 1000 requests per day and prevents excessive usage over a longer period

AnonBurstRateThrottle - For Anonymous Users

Maximum 20 requests per minute and stricter limits for non-logged in users and prevents also bots from scraping per data

User Type	Short-term Limit	Long-term Limit
Logged-in	60 requests/minute	1,000 requests/day
Anonymous	20 requests/minute	(only short-term)

And all this matters because of:

Problem	Solution
Bots scraping data	Anonymous users limited to 20/min
Users spamming buttons	Burst limit prevents 100 clicks/second
API overload	Daily limit ensures fair usage
Server crashing	Throttling prevents DoS attacks

Utils.py: this file contains small reusable helper functions that perform common tasks throughout our applications

send_notification_email() - Sends Emails

Sends html emails to users using Django templates and automatically creates plain text version from html like:

```
# When a trip is confirmed
send_notification_email(
    user=passenger,
    subject="Your trip is confirmed!",
    template_name="emails/booking_confirmation.html",
    context={"trip": trip, "date": "2024-03-15"}
)
```

format_distance() - Formats Distance

By this function our site web will be able to show the number of distance in m or km to the user

```
format_distance(0.5)    → "500 m"
format_distance(1.2)    → "1.2 km"
format_distance(150.75) → "150.8 km"
```

format_price() - Formats Algerian Dinar

By this function our site web will be able to show the price of the trip for the user in dinar only

```
format_price(500)      → "500.00 DA"
format_price(1250.5)   → "1,250.50 DA"
format_price(10000)    → "10,000.00 DA"
```

calculate_trip_duration() - Estimates Trip Time

By this function our site web will be able to show the time and the duration of the trip to the user in h or min

```
calculate_trip_duration(120)    → 1:30:00 (1.5 hours)
calculate_trip_duration(40)     → 0:30:00 (30 minutes)
calculate_trip_duration(300, 100) → 3:00:00 (3 hours at 100 km/h)
```

generate_unique_code() - Creates Random Codes

By this function create a random code for every trip and it will be unique and will be used for that trip only

```
generate_unique_code()      → "A7K9B2M1"
generate_unique_code(6)     → "X3P9L2"
generate_unique_code(10)    → "R5T8W2M4K9"
```

As all those functions work together and will be functioned at the same time we will get this as booking as an example

```
# Creating a new booking
def create_booking(request):
    booking = Reservation.objects.create(
        passager=request.user,
        trajet=trip,
        reference=generate_unique_code(), # "X7K9B2M1"
        price=trip.price
    )

    # Send confirmation email
    send_notification_email(
        user=request.user,
        subject="Booking Confirmed",
        template_name="emails/booking.html",
        context={
            "trip": trip,
            "distance": format_distance(trip.distance), # "120.5 km"
            "duration": calculate_trip_duration(trip.distance), # "1:30:00"
            "price": format_price(trip.price) # "1,500.00 DA"
        }
    )
```

Function	Without Helper	With Helper
Email	5 lines + HTML processing	1 line call
Distance	Manual if/else everywhere	Consistent formatting
Price	Remember format pattern	Always correct format
Duration	Complex math each time	Simple call
Codes	Random logic scattered	Centralized generation

Validators.py : this file contains validation functions that check if user inputs meets specific requirements before saving to the database

Phone number check: must start with 0 or +213, must have 5 6 7 as a second digit and must have exactly 8 more digits

Region check: the input must be one of the 48 Algerian wilayas

Date check: date must be today pr on the future not in the past

Price check: price must be between 100DA and 50,000DA and in decimal

Seat count check: number of seats must be between 1 and 8

```

from django.db import models
from .validators import (
    validate_algerian_phone,
    validate_wilaya_name,
    validate_future_date,
    validate_price_range,
    validate_seats_count
)

```

```

class Trajet(models.Model):
    ville_depart = models.CharField(
        max_length=50,
        validators=[validate_wilaya_name] # 👉 Auto-validates
    )
    ville_arrivee = models.CharField(
        max_length=50,
        validators=[validate_wilaya_name] # 👉 Auto-validates
    )
    date = models.DateField(
        validators=[validate_future_date] # 👉 Auto-validates
    )
    prix = models.DecimalField(
        max_digits=10,
        decimal_places=2,
        validators=[validate_price_range] # 👉 Auto-validates
    )
    places = models.IntegerField(
        validators=[validate_seats_count] # 👉 Auto-validates
    )

```

```

class User(models.Model):
    phone = models.CharField(
        max_length=20,
        validators=[validate_algerian_phone] # 👉 Auto-validates
    )

```

Validator	Rule	Example Error
Phone	Must be Algerian mobile	"Invalid phone format"

Wilaya	Must be one of 48 regions	"Not an Algerian wilaya"
Date	Must be future	"Date must be in the future"
Price	Between 100-50,000 DA	"Price out of range"
Seats	Between 1-8	"Seats must be 1-8"

SAIDI_MAHMOUD_2CP7